

LAPORAN PENGEMBANGAN APLIKASI BOOKSHELF

1. Gambar Umum Aplikasi

Bookshelf merupakan aplikasi pengelola koleksi buku pengguna secara privat. Bookshelf dirancang untuk mencatat dan merekap daftar buku yang telah dibaca oleh pengguna dalam jangka waktu tertentu. Di samping itu, Bookshelf juga mampu merekap progres pembacaan buku serta memberi rating-review pada buku yang telah dibaca. Data tersimpan lokal di IsarDB kemudian dapat disinkronkan ke backend agar koleksi buku bisa diakses dari mana saja.

2. Arsitektur yang kami gunakan

Aplikasi Bookshelf kami bangun berbasis offline-first serta membagi task antara perangkat user pada frontend serta sever di backend. Adapun arsitektur sistem dari aplikasi Bookshelf sebagai berikut:

Komponen	Framework	Catatan
Mobile Front-end	Flutter (Dart SDK)	Manajemen UI/UX dan State Management
Local Database	IsarDB	Penyimpanan model Buku, Progres, dan Review secara lokal (<i>offline</i>)
Back-end API	Go (Chi)	Penyedia REST API untuk manajemen data terpusat
Server Database	PostgreSQL	Penyimpanan data persisten di tingkat <i>cloud server</i>
HTTP Client	Dio / Http (Package)	Media komunikasi data dari Flutter menuju Flask API

3. Kendala yang kami temukan

Saat mengembangkan aplikasi Bookshelf kami menemukan kendala yang cukup banyak, berikut di antaranya:

[Database Lokal & Mode *Offline-First*]

- **Kendala:** Data buku yang ditambah, diedit, atau dihapus saat *offline* justru hilang tertimpa oleh data lama dari server saat aplikasi kembali *online* (terjadi *data loss*).
- **Solusi:** Mengubah struktur penanda dari sekadar `isSynced` menjadi `syncAction` (`none`, `add`, `edit`, `delete`). Dilakukan agar **tidak** menimpa baris data lokal yang belum dikirim ke server.

[Saat Integrasi API & Fungsionalitas]

- **Kendala:** Tombol hapus buku gagal karena memanggil fungsi `deleteBook` (tunggal), sedangkan *endpoint* API hanya menerima `deleteBooks` (jamak/berkelompok).
- **Solusi:** Menyesuaikan kode di UI `BookDetailPage` agar membungkus ID buku tunggal tersebut ke dalam sebuah *List/Array* (`[_book.id]`) saat memanggil API.

- **Kendala:** Logika *auto-login* (jika sesi sudah ada) belum berjalan mulus, menyisakan peringatan *unused variables* di halaman Login.
- **Solusi:** Memindah gerbang logika *auto-login* ke titik paling awal (*main.dart*). Aplikasi mengecek status di *Splash Screen* dan otomatis mengarahkan ke *HomePage* jika token tersedia.

[Saat *Build & Kompilasi (.apk)*]

- **Kendala:** Error *Namespace not specified* dan *android:attr/Star not found* akibat plugin lawas (*isar_flutter_libs*) mengunci kompilasi pada Android SDK jadul (API 30), yang bentrok dengan *library* kamera *mobile_scanner* (API 35).
- **Solusi:** Memasang *override script* pada *android/build.gradle.kts* tingkat *root* untuk memaksa semua *plugin* ikut memakai SDK versi terbaru (API 35) dan memberikan *namespace* otomatis.

[Saat pembuatan Rest API]

- Konversi Model

Kendala: Class suatu objek seperti Buku dari Database dengan Buku yang berasal dari JSON memiliki format yang berbeda. Buku yang berasal dari database bertipe data khusus. Sejauh ini, data yang berasal dari *HttpRequest* bisa langsung diterima Buku Format Database. Akan tetapi, kita tidak bisa langsung assign Buku format database ke response HTTP karena akan menghasilkan format beda.

Solusi: buat semacam converter (*/backend/internal/converter/type_database.go*)

- Deploy REST API

Kendala: Deploy server REST API secara manual melalui VPS cukup rumit dan menantang. Kita perlu install dependencies secara manual seperti *golang*, *PostgreSQL*, *sqlc*, dan *goose*. Akan tetapi, ini tidak menimbulkan masalah signifikan. Perbedaan versi saat development dan produksi belum menjadi masalah karena perbedaan versi yang belum terlalu jauh--tentu saat ingin membangun aplikasi nyata, kesesuaian versi antara produksi dan development harus diperhatikan agar tidak error saat mengembangkan aplikasi lebih jauh.

- HTTP Request

Kendala: Proyek ini mengutamakan aplikasi yang berjalan secara offline dan sinkronisasi data saat online. Kami memutuskan untuk menyediakan REST API yang langsung menangani data dalam jumlah banyak. Dengan kata lain, endpoint akan menerima list of item. Permasalahannya terletak pada HTTP Request DELETE. DELETE tidak bisa menerima request body.

Solusi: Menggunakan POST pada endpoint yang seharusnya bertindak menghapus.